

objective of this example is to introduce the steps, more than the actual output.

```
using Flux
actual(x) = 2x + 3
x_train, x_test = hcat(0:5...), hcat(6:10...)
y_train, y_test = actual.(x_train), actual.(x_test)
```

Here, the function is coded, and the training and test data are built. Usually, in a real-time scenario, the training and test data will be gathered and cleaned with expert assistance. As we are demonstrating the capability, we have generated the data using the function. The output is as shown below:

```
([3 5 ... 11 13], [15 17 ... 21 23])
```

Let's build a model:

```
# Building the model
model = Dense(1, 1)
```

The model variable will now contain the fields *weight* and *bias*. The values of these can be obtained by *model*. *weight* and *model.bias*. The models here are the predictive functions.

```
predict = Dense(1, 1)
```

Along with these values, the *Dense(1,1)* will also implement the function $\sigma(Wx+b)$. It can be noted that Julia enables naming of functions with Greek letters as well, to match with the equations in the research papers, etc. For example, the following is a Sigmoid function:

```
 $\sigma(1)$ 
0.7310585786300049
```

If you try the predictions without training, it will obviously make incorrect predictions. So, the next step is to provide a loss function. The loss function informs the Flux how well the model is doing. It does so by computing the cumulative difference between the actual and the predicted values.

```
loss(x, y) = Flux.Losses.mse(predict(x), y)
loss(x_train, y_train)
```

In the aforementioned code, MSE stands for *Mean Squared Error*. The objective of training is to reduce the loss with training.

Improving the prediction can be carried out with the *train!* function. This function takes the loss function, the parameters, data and optimiser as inputs to perform the training.

```
using Flux: train!
opt = Descent()
data = [(x_train, y_train)]
parameters = params(predict)
train!(loss, parameters, data, opt)
```

Loop the training by iterating through the data set with the following code snippet:

```
for epoch in 1:200
    train!(loss, parameters, data, opt)
end
loss(x_train, y_train)
```

After the iterative process, the value of loss can be observed as a smaller value. In your implementation, the actual value might differ. But the point here is that it will be a very small one (near zero).

```
0.0041534975f0
```

The results can be verified as follows:

```
predict(x_test)
1×5 Matrix{Float32}:
 15.124  17.1438  19.1635  21.1833  23.203

y_test
1×5 Matrix{Int64}:
 15  17  19  21  23
```

It can be observed from the aforementioned snippet that the predicted values are almost equal to the actual output. This proves that our model has trained well to predict the output of the function that we have given in the beginning.

The models can be constructed by combining various layers (chaining), as shown below:

```
model2 = Chain(
    Dense(10, 5,  $\sigma$ ),
    Dense(5, 2),
    softmax)
```

Taking gradients

Computing gradients is at the core of many of the ML approaches. Flux enables this computation process to be very effective and optimal. The differentiation can be